

UNIVERSIDADE FEDERAL DO PARANÁ
PÓS-GRADUAÇÃO EM INTELIGÊNCIA ARTIFICIAL APLICADA
IAA006 - Arquitetura de Dados

Trabalho Técnico / Atividade 02

Integrantes:

Amanda Isabelly Neumann Padilha
Gustavo Gomes Marcondes
Helen Sara Teixeira
Igor Nathan Lobato
Jean Roberto Reinert
Matheus Henrique Miranda

Curitiba 2026

2 Melhore uma base de dados ruim

Enunciado

Escolha uma base de dados pública para problemas de classificação, disponível ou com origem na UCI Machine Learning. Use o mínimo de intervenção para rodar a SVM e obtenha a matriz de confusão dessa base. O trabalho começa aqui, escolha as diferentes tarefas discutidas ao longo da disciplina, para melhorar essa base de dados, até que consiga efetivamente melhorar o resultado. Considerando a acurácia para bases de

dados balanceadas ou quase balanceadas, se o percentual da acurácia original estiver em até 85%, a meta será obter 5%. Para bases com mais de 90% de acurácia, a meta será obter a melhora em pelo menos 2 pontos percentuais (92% ou mais). Nessa atividade deverá ser entregue o script aplicado (o notebook e o PDF correspondente).

Seleção da Base na UCI

Selecionamos a base de Doenças Cardíacas (Heart Disease), própria para classificação, disponível em:

<https://archive.ics.uci.edu/dataset/45/heart+disease>

Como as classes variam de 0 (Sem doença) a 4 (Diferentes tipos de doença), nós vamos nos limitar a determinar apenas se 0 - Tem Doença ou 1 - Não tem doença. Dessa forma, não corremos o risco de prejudicar o modelo com classes que possuem poucas ocorrências.

In [1]: *# Importação das Bibliotecas*

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn import svm
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score
```

In [2]: *# Download da base*

```
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/heart-disease/p
colunas = ['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg',
           'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target']
df = pd.read_csv(url, names=colunas, na_values='?')
```

In [3]: `df.head(10)`

Out[3]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
--	-----	-----	----	----------	------	-----	---------	---------	-------	---------	-------	----	------	--------

0	63.0	1.0	1.0	145.0	233.0	1.0	2.0	150.0	0.0	2.3	3.0	0.0	0	0
1	67.0	1.0	4.0	160.0	286.0	0.0	2.0	108.0	1.0	1.5	2.0	3.0	0	0
2	67.0	1.0	4.0	120.0	229.0	0.0	2.0	129.0	1.0	2.6	2.0	2.0	0	0
3	37.0	1.0	3.0	130.0	250.0	0.0	0.0	187.0	0.0	3.5	3.0	0.0	0	0
4	41.0	0.0	2.0	130.0	204.0	0.0	2.0	172.0	0.0	1.4	1.0	0.0	0	0
5	56.0	1.0	2.0	120.0	236.0	0.0	0.0	178.0	0.0	0.8	1.0	0.0	0	0
6	62.0	0.0	4.0	140.0	268.0	0.0	2.0	160.0	0.0	3.6	3.0	2.0	0	0
7	57.0	0.0	4.0	120.0	354.0	0.0	0.0	163.0	1.0	0.6	1.0	0.0	0	0
8	63.0	1.0	4.0	130.0	254.0	0.0	2.0	147.0	0.0	1.4	2.0	1.0	0	0
9	53.0	1.0	4.0	140.0	203.0	1.0	2.0	155.0	1.0	3.1	3.0	0.0	0	0

Intervenção mínima na base

Apenas intervenções pra permitir rodar o SVM baseline

```
In [4]: ## Removemos as linhas com NaN para o algoritmo não quebrar
df_baseline = df.dropna().copy()
```

```
In [5]: # Colapso do target
# Como comentado acima, se for 0, continua 0 (Saudável). Se for maior que 0, vir
df_baseline['target'] = df_baseline['target'].apply(lambda x: 1 if x > 0 else 0)

df_baseline.head(10)
```

```
Out[5]:
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	th
0	63.0	1.0	1.0	145.0	233.0	1.0	2.0	150.0	0.0	2.3	3.0	0.0	0
1	67.0	1.0	4.0	160.0	286.0	0.0	2.0	108.0	1.0	1.5	2.0	3.0	1
2	67.0	1.0	4.0	120.0	229.0	0.0	2.0	129.0	1.0	2.6	2.0	2.0	1
3	37.0	1.0	3.0	130.0	250.0	0.0	0.0	187.0	0.0	3.5	3.0	0.0	1
4	41.0	0.0	2.0	130.0	204.0	0.0	2.0	172.0	0.0	1.4	1.0	0.0	1
5	56.0	1.0	2.0	120.0	236.0	0.0	0.0	178.0	0.0	0.8	1.0	0.0	1
6	62.0	0.0	4.0	140.0	268.0	0.0	2.0	160.0	0.0	3.6	3.0	2.0	1
7	57.0	0.0	4.0	120.0	354.0	0.0	0.0	163.0	1.0	0.6	1.0	0.0	1
8	63.0	1.0	4.0	130.0	254.0	0.0	2.0	147.0	0.0	1.4	2.0	1.0	1
9	53.0	1.0	4.0	140.0	203.0	1.0	2.0	155.0	1.0	3.1	3.0	0.0	1

Rodando a SVM (Dados não tratados)

```
In [6]: # Separar X (Atributos) e Y (Classe)
X = df_baseline.drop('target', axis=1)
y = df_baseline['target']
```

```
In [7]: # Divisão de Treino e Teste
# Como as classes são balanceadas no dataset, optamos por fazer uma divisão simp
# utilizando 25% para teste conforme exemplo do professor (cancer de mama)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, strati
```

```
In [8]: # Treinamento da SVM (Com dados ruins/não padronizados)
modelo_orig = svm.SVC()
modelo_orig.fit(X_train, y_train)
```

```
Out[8]:
```

▼ SVC ⓘ ?

► Parameters

Matriz Confusão e Métricas Baseline (Dados não Tratados)

```
In [9]: y_pred = modelo_orig.predict(X_test)

print('--- Matriz de Confusão (Baseline) ---')
print(confusion_matrix(y_test, y_pred))
print('\n--- Relatório de Classificação ---')
print(classification_report(y_test, y_pred))
print(f'\nAcurácia Original: {accuracy_score(y_test, y_pred):.2%}')
```

```
--- Matriz de Confusão (Baseline) ---
[[33  7]
 [17 18]]

--- Relatório de Classificação ---
```

	precision	recall	f1-score	support
0	0.66	0.82	0.73	40
1	0.72	0.51	0.60	35
accuracy			0.68	75
macro avg	0.69	0.67	0.67	75
weighted avg	0.69	0.68	0.67	75

Acurácia Original: 68.00%

Matriz Confusão:

- Verdadeiros Negativos: 33 pessoas saudáveis identificadas corretamente pelo modelo.
- Falsos Positivos: 7 pessoas identificadas pelo modelo como doentes são, na verdade, saudáveis.
- Falsos Negativos: 17 pessoas previstas como saudáveis pelo modelo, na verdade eram doentes
- Verdadeiros Positivos: 18 pessoas doentes corretamente identificadas pelo modelo.

Acurácia

68%

Tratamento de Dados

Carregamento dos dados

```
In [10]: # Importações de bibliotecas
from sklearn.impute import SimpleImputer
```

```

from sklearn.preprocessing import MinMaxScaler

# Carregamento inicial (igual ao anterior)
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/heart-disease/p
colunas = ['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg',
           'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target']
df = pd.read_csv(url, names=colunas, na_values='?')

df['target'] = df['target'].apply(lambda x: 1 if x > 0 else 0)

```

Procedimento 1: Limpeza e Preenchimento dos Dados

```

In [11]: # Quantidade de valores nulos por coluna
print(df.isna().sum())

# Linhas que possuem o problema
display(df[df.isna().any(axis=1)])

```

```

age      0
sex      0
cp       0
trestbps 0
chol     0
fbs      0
restecg  0
thalach  0
exang    0
oldpeak  0
slope    0
ca       4
thal     2
target   0
dtype: int64

```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca
87	53.0	0.0	3.0	128.0	216.0	0.0	2.0	115.0	0.0	0.0	1.0	0.0
166	52.0	1.0	3.0	138.0	223.0	0.0	0.0	169.0	0.0	0.0	1.0	NaN
192	43.0	1.0	4.0	132.0	247.0	1.0	2.0	143.0	1.0	0.1	2.0	NaN
266	52.0	1.0	4.0	128.0	204.0	1.0	0.0	156.0	1.0	1.0	2.0	0.0
287	58.0	1.0	2.0	125.0	220.0	0.0	0.0	144.0	0.0	0.4	2.0	NaN
302	38.0	1.0	3.0	138.0	175.0	0.0	0.0	173.0	0.0	0.0	1.0	NaN

Verificamos que 6 linhas da base estão com valores nulos e sendo descartadas do treinamento. Com a inclusão delas, nossa expectativa é melhorar a acurácia. Vamos preencher os valores nulos pela Moda, visto que as variáveis são categóricas (ca - tipo de dor no peito e thal - Talassemia).

```

In [12]: imputer = SimpleImputer(strategy='most_frequent')
df_imputed = pd.DataFrame(imputer.fit_transform(df), columns=df.columns)

```

```
In [13]: # Conferindo o resultado:
print(df_imputed.isna().sum())
```

```
age      0
sex      0
cp       0
trestbps 0
chol     0
fbs      0
restecg  0
thalach  0
exang    0
oldpeak  0
slope    0
ca       0
thal     0
target   0
dtype: int64
```

Procedimento 2: Codificação dos Dados

Verificamos na descrição do dataset que algumas variáveis são categóricas: cp, restecg, slope, thal. Para essas, aplicaremos a técnica de one-hot-encoding

```
In [14]: for col in ['cp', 'restecg', 'slope', 'thal']:
print(f"Valores únicos na coluna '{col}': {df_imputed[col].unique()}")
```

```
Valores únicos na coluna 'cp': [1. 4. 3. 2.]
Valores únicos na coluna 'restecg': [2. 0. 1.]
Valores únicos na coluna 'slope': [3. 2. 1.]
Valores únicos na coluna 'thal': [6. 3. 7.]
```

```
In [15]: colunas_categoricas = ['cp', 'restecg', 'slope', 'thal']
df_tratado = pd.get_dummies(df_imputed, columns=colunas_categoricas, drop_first=

# Mostrando o dataset já codificado com as variáveis novas.
display(df_tratado.head())
```

	age	sex	trestbps	chol	fbs	thalach	exang	oldpeak	ca	target	cp_2.0	cp_3.0
0	63.0	1.0	145.0	233.0	1.0	150.0	0.0	2.3	0.0	0.0	False	False
1	67.0	1.0	160.0	286.0	0.0	108.0	1.0	1.5	3.0	1.0	False	False
2	67.0	1.0	120.0	229.0	0.0	129.0	1.0	2.6	2.0	1.0	False	False
3	37.0	1.0	130.0	250.0	0.0	187.0	0.0	3.5	0.0	0.0	False	True
4	41.0	0.0	130.0	204.0	0.0	172.0	0.0	1.4	0.0	0.0	True	False

Procedimento 3: Normalização dos Dados

```
In [16]: resumo_antes = X[['age', 'chol', 'thalach', 'oldpeak']].describe().loc[['min', '
display(resumo_antes)
```

	age	chol	thalach	oldpeak
min	29.000000	126.000000	71.000000	0.000000
max	77.000000	564.000000	202.000000	6.200000
mean	54.542088	247.350168	149.599327	1.055556

Verificamos que as variáveis age, chol, thalach e oldpeak possuem escalas totalmente diferentes, o que pode impactar no treinamento do modelo. Por isso, usaremos da estratégia de interpolação linear visto que os dados não possuem outliers extremos nem outras circunstâncias que justifiquem outra estratégia.

```
In [17]: # Já vamos separar atributos e target
X = df_tratado.drop('target', axis=1)
y = df_tratado['target']

# Aplicação da interpolação linear
scaler = MinMaxScaler()
X_scaled = pd.DataFrame(scaler.fit_transform(X), columns=X.columns)
```

```
In [18]: # Verificando os resultados:

resumo_apos = X_scaled[['age', 'chol', 'thalach', 'oldpeak']].describe().loc[['m
display(resumo_apos)
```

	age	chol	thalach	oldpeak
min	0.000000	0.000000	0.000000	0.000000
max	1.000000	1.000000	1.000000	1.000000
mean	0.529978	0.275555	0.600055	0.167678

Treinamento com Dados Tratados

```
In [19]: # Treinando igual havíamos treinado inicialmente no baseline
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.25,
modelo_tratado = svm.SVC()
modelo_tratado.fit(X_train, y_train)
y_pred = modelo_tratado.predict(X_test)
```

Resultados e Conclusão

```
In [20]: print('--- Matriz de Confusão (Após Tratamento) ---')
print(confusion_matrix(y_test, y_pred))
print('\n--- Relatório de Classificação ---')
print(classification_report(y_test, y_pred))
print(f'\nAcurácia Melhorada: {accuracy_score(y_test, y_pred):.2%}')
```

--- Matriz de Confusão (Após Tratamento) ---

```
[[36  5]
 [ 2 33]]
```

--- Relatório de Classificação ---

	precision	recall	f1-score	support
0.0	0.95	0.88	0.91	41
1.0	0.87	0.94	0.90	35
accuracy			0.91	76
macro avg	0.91	0.91	0.91	76
weighted avg	0.91	0.91	0.91	76

Acurácia Melhorada: 90.79%

Conclusão: Após aplicação das técnicas de Preenchimento, Codificação e Normalização a acurácia da SVM saltou de 68% para 90%.